



Big-Step 증명나무 합성으로 분석기 골탕먹이기

정원준

2026년 5월 15일

ROPAS@SNU

목차

정적 분석기의 부정확성

프로그램 합성과 증명나무 합성

증명나무 합성 디테일

결과

앞으로 할 일

정적 분석기의 부정확성

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?

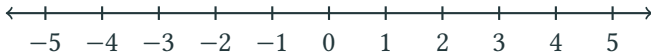
정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?
- 완벽하게 알 수는 없음!

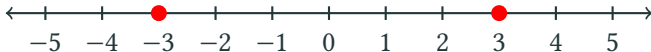
넓게 어림잡기

```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



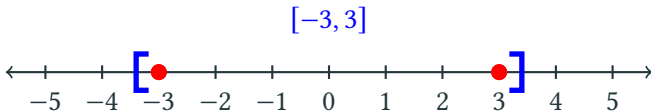
넓게 어림잡기

```
if x > 0:  
    x = 3  
else:  
    x = -3  
print(x)
```



넓게 어림잡기

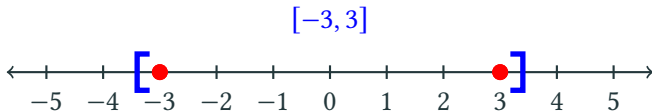
```
if x > 0:  
    x = 3  
else:  
    x = -3  
print(x)
```



공격 프로그램의 예시

공격 프로그램의 예시

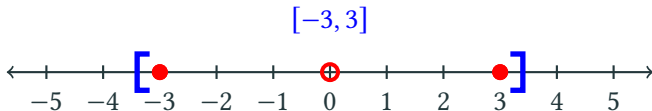
```
if x > 0:  
    x = 3  
else:  
    x = -3  
print(x)
```



공격 프로그램의 예시

```
if x > 0:  
    x = 3  
else:  
    x = -3  
y = 10 / x
```

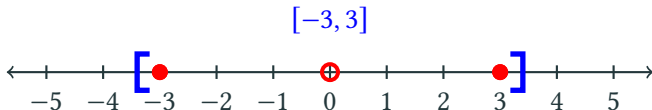
false alarm 발생



공격 프로그램의 예시

```
if x > 0:  
    x = 3  
else:  
    x = -3  
y = 10 / x
```

false alarm 발생



이런 공격을 일반적인 분석기에 대해서 하는 것이 목표

공격의 기대효과

공격의 기대효과

- 분석기의 약점을 파악하고 강화하는 가이드라인으로 사용

공격의 기대효과

- 분석기의 약점을 파악하고 강화하는 가이드라인으로 사용
- 분석이 잘 안 되는 코드 조각을 삽입해서 난독화로 사용

프로그램 합성과 증명나무 합성

프로그램 합성

- 주어진 성질을 만족하는 프로그램을 자동으로 생성!
- "입력이 1, 2, 3, 4일 때 각각 1, 3, 6, 10이 나오는 프로그램을 만들어 줘" -> (코드)
- "입력이 x 이고 출력이 y 일 때, $y^2 \leq x < (y + 1)^2$ 가 되는 프로그램을 만들어 줘" -> (코드)

증명나무 합성

- ”분석기가 false alarm을 내는 프로그램을 만들어 줘”
- bottom-up enumeration을 어떻게 흉내낼 수 있는지 설명

증명나무 합성 디테일

증명나무 합성 디테일

- size 정의가 왜 중요한가, 2차원 size 순서 정렬
- 프로그램 합성과 다르게 짝짜꿍이 잘 맞아야 됨
- environment 제한
- proof tree가 아닌 일반 코드 조각도 합성해야 함
- pruning 방법, "분석기 존중"의 정의, 프로그램 합성에서의 pruning과 어떻게 다른지

결과

결과

- 첫번째 결과 설명: while 0 처리 안됨
- 두번째 결과 설명: interval widening
- 분석기가 굉장히 약한 걸로 시작해서, 합성기와 평행하게 같이 강해지기

앞으로 할 일

- ”좋은 proof tree 조각” 평가 휴리스틱
- bottom up은 너무 많은 코드/증명 조각을 생성함. 좋은 쪽으로 많이 생성하려면?
- ”좋은 공격”이 뭔지 계속 다듬으면서 생각하기
- pruning
- llm guide
- ”비슷한 공격”들을 많이 제시해주는 건 의미가 없음. 비슷한 걸 거르고 다양하게 결과를 내주려면?

감사합니다